

Créé par le Centre d'Excellence Cyberdéfense Aérospatiale (CEC)

Guide des tests **WAC (Windows Artefacts Collector)**

Logiciel de prélèvement d'artefacts Windows
pour l'investigation numérique



ARTEFACTS
SYSTÈME



FICHIERS
UTILISATEUR



PREUVES
NUMÉRIQUES

UN OUTIL AU SERVICE
DE L'INVESTIGATION
NUMÉRIQUE

WAC — Windows Artefacts Collector

[Guide des tests](#)

Version du logiciel	1.3.1
Édition du document	septembre 2026
Éditeur	Centre d'Excellence Cyberdéfense Aérospatiale (CEC) École de l'air et de l'espace, base aérienne 701, Salon-de-Provence

Ce document explique comment lancer à la main chacun des tests de WAC, ce que chacun prouve et ne prouve pas, et comment lire un échec. Il s'adresse à qui modifie le code ou veut vérifier une version avant de l'employer. Les durées et volumes cités ont été relevés sur la machine virtuelle de test et sur une collecte réelle; ce sont des ordres de grandeur.

Table des matières

1 Principes	3
1.1 Ce qu'un test de WAC doit prouver	3
1.2 Trois règles	3
2 Préparer l'environnement	4
2.1 Le poste Linux	4
2.2 Construire WAC et les harnais	4
2.3 La machine virtuelle de test	5
3 Le cycle de test dans la machine virtuelle	6
3.1 Lancer un cycle	6
3.2 Contrôler les sorties	6
3.3 Comparer deux passages	7
4 Les harnais de format	8
4.1 Empreintes : sha_test	8
4.2 Décompression : lznt1_test et xpress_test	8
4.3 Messages d'événements : wevt_test et event_messages_test	8
4.4 Journaux : evt_x_test	9
4.5 Consigne : consigne_test	9
4.6 Lecture brute : raw_hive_test	9
4.7 Signatures : authenticode_test	10
5 Les harnais de robustesse	11
5.1 Le principe	11
5.2 Raccourcis : lnk_test	11
5.3 Listes de sauts et Prefetch : parsers_test	11
5.4 Référence	12
6 Recouper sur une collecte réelle	13
7 Lire un échec	14

7.1	Codes de sortie	14
7.2	Localiser une lecture hors tampon	14
7.3	Une boucle sans fin	15

Principes

1.1 Ce qu'un test de WAC doit prouver

WAC produit des fichiers JSON. Qu'ils soient valides ne prouve presque rien : un champ lu au mauvais décalage, une date interprétée dans le mauvais fuseau, un nom décodé dans la mauvaise page de code donnent un JSON parfaitement valide — et faux. Les défauts les plus coûteux relevés dans WAC étaient tous de cette sorte : ils ne se voyaient qu'en confrontant une valeur à une autre source.

Les tests se rangent donc en trois familles, qui ne répondent pas à la même question :

Le cycle dans la machine virtuelle (chapitre 3) : WAC s'exécute-t-il jusqu'au bout sur un vrai Windows, et ses sorties sont-elles cohérentes entre elles ? Les *contrôles croisés* confrontent une valeur calculée par WAC à une valeur indépendante de la même collecte.

Les harnais de format (chapitre 4) : un décodeur donne-t-il exactement le résultat de la référence — l'outil de Microsoft, une norme publiée, une autre implémentation ?

Les harnais de robustesse (chapitre 5) : un analyseur lit-il en dehors de son tampon, ou boucle-t-il sans fin, sur un fichier tronqué ou forgé ?

1.2 Trois règles

1. **Un correctif s'accompagne du test qui aurait pris le défaut.** Et ce test se vérifie dans les deux sens : il passe sur la sortie corrigée, et il échoue quand on rejoue le défaut.
2. **Après tout changement, comparer au passage précédent,** et pas seulement lire les pastilles vertes : un artefact passé de 1 522 entrées à 573 n'allume aucun voyant (section 3.3).
3. **Une donnée réelle ne quitte pas le poste de test.** Une collecte réelle contient des données personnelles : elle se traite en local, ses copies temporaires s'effacent après usage, et rien n'en entre dans le dépôt (`vmtest/results/` est ignoré par git).

Préparer l'environnement

2.1 Le poste Linux

Tout se construit sur Linux, par compilation croisée MinGW-w64; les harnais tournent soit nativement, soit sous wine, soit dans la machine virtuelle.

```
sudo apt install g++-mingw-w64-x86-64 wine64 python3 python3-pip
pip install olefile          # lecteur OLE indépendant, pour les recoupements
```

2.2 Construire WAC et les harnais

```
./build-windows.sh --test
```

Produit	Rôle
build-windows/WAC.exe	l'outil de collecte
build-windows/raw_hive_test.exe	lecteur NTFS brut, utilisé par le cycle VM
build-windows/tests/lnk_test.exe	robustesse de l'analyseur de raccourcis
build-windows/tests/parsers_test.exe	robustesse des listes de sauts et des Prefetch
build-windows/tests/propsys.dll, offreg.dll	substituts pour wine, qui n'a pas ces deux bibliothèques

Attention

Les deux substituts sont volontairement dans build-windows/tests/, à l'écart de WAC.exe. Placés à côté de lui, ils seraient chargés à la place des vraies bibliothèques — Windows cherche d'abord dans le dossier du programme — et WAC ne lirait plus aucune ruche, sans erreur visible. Ne jamais copier le dossier tests/ sur un support de collecte.

La compilation doit se terminer sans aucun avertissement : les options -Wall -Wextra sont actives, et un avertissement a déjà signalé de vrais défauts.

Les autres harnais (chapitre 4) se construisent chacun par une commande donnée dans son en-tête et reprise dans ce guide; plusieurs compilent nativement sous Linux.

2.3 La machine virtuelle de test

La VM est un Windows 11 piloté par le *qemu-guest-agent* : aucune interaction, aucun réseau, les commandes s'exécutent en NT AUTHORITY\SYSTEM. Sa création et ses prérequis sont décrits dans `vmtest/README_FR.md`; en résumé :

```
cd vmtest
ISO_WIN=~/.Téléchargements/Win11_25H2_French_x64_v2.iso ./create-vm.sh # une fois
python3 qga.py ping # l'agent répond ?
python3 qga.py run --shell "dir C:\"" # exécuter une commande
python3 qga.py write <hôte> <invité> # envoyer un fichier
python3 qga.py read <invité> <hôte> # rapatrier un fichier
```

À retenir

Une commande lancée par `qga.py run` est coupée au bout de 30 minutes. Pour un test plus long (par exemple les Prefetch d'une machine réelle, section 5.3), le lancer en tâche détachée dans la VM et suivre son journal.

Le cycle de test dans la machine virtuelle

3.1 Lancer un cycle

```
cd vmtest
./run-wac-test.sh --build      # recompile, puis cycle complet
./run-wac-test.sh             # cycle complet avec les binaires déjà construits
./run-wac-test.sh --raw-only  # seulement la validation de la lecture brute
```

Le cycle enchaîne : envoi de WAC.exe et raw_hive_test.exe dans la VM, validation de l'extraction brute d'une ruche (elle doit être refusée par reg load tant qu'elle n'est pas réparée, puis acceptée), exécution de WAC, puis rapatriement, dans le dossier vmtest/results/<horodatage>/, de :

- run.log — la sortie console de la collecte, qui se termine par END, Time elapsed quand WAC est allé au bout;
- raw.log — la trace du lecteur NTFS;
- les JSON des artefacts;
- exhibits/ — le manifeste de consigne, son sceau et la liste des fichiers réellement présents dans la consigne.

3.2 Contrôler les sorties

```
python3 check-json.py results/<horodatage>
```

Chaque ligne commence par une coche verte (contrôle passé), une croix rouge (défaut) ou un signe « passer » (contrôle sans objet : l'artefact est absent ou vide). Le code de sortie vaut 0 si aucun contrôle n'a échoué. Au-delà de la validité JSON et de la forme des chemins, le script confronte des sources indépendantes :

Contrôle croisé	Ce qu'il a déjà trouvé
empreinte d'un Prefetch / suffixe de son nom de fichier	65 empreintes fausses sur 270 (hexadécimal sans remplissage)
nom de l'exécutable d'un Prefetch / début du nom de fichier	un nom lu 8 octets trop tôt (« \x11 » partout)

Contrôle croisé	Ce qu'il a déjà trouvé
chemins \VOLUME{guid} d'un Prefetch résolus	20 066 chemins sur 20 066 non résolus
couples X / XUtc	double décalage horaire
aucune session antérieure au démarrage	décalages de dates, heure de démarrage en retard de 3,5 s
date ShimCache / date NTFS du manifeste	une date UTC traitée comme locale (-2 h)
entrées DestList / raccourcis des listes de sauts	les 15 premières entrées de chaque liste perdues
nom de machine des événements / OperatingSystem.json	contrôle du décodage BinXML
identifiants d'enregistrement uniques par journal	relecture d'un chunk périmé
sceau / empreinte réelle du manifeste	retouche du manifeste
fichiers de la consigne / contenu du manifeste	209 pièces ajoutées après le scellement

La liste complète, avec l'histoire de chaque contrôle, est dans `vmtest/README_FR.md`.

3.3 Comparer deux passages

Une pastille verte ne dit pas si une donnée a disparu. Après tout changement, comparer la nouvelle collecte à la précédente : nombre d'entrées de chaque artefact, et valeurs perdues ou gagnées. Deux précautions :

- une collecte change d'elle-même d'un passage à l'autre : horodatages, nouveaux événements, fichiers Prefetch mis à jour, empreinte de \$MFT. On compare donc les éléments *appariés* — par exemple un Prefetch par son empreinte de chemin et sa dernière exécution — en écartant les dates de la copie extraite ;
- une valeur « perdue » peut n'être qu'un champ vide qui n'est plus émis, ou une clé renommée : normaliser avant de conclure.

CHAPITRE 4

Les harnais de format

Chacun confronte un décodeur de WAC à une référence. Tous sont exclus de la compilation de WAC par le suffixe `_test.cpp`.

4.1 Empreintes : `sha_test`

Confronte SHA-1 et SHA-256 aux vecteurs publiés de FIPS 180-4, et aux longueurs 55 à 128 octets, qui exercent le remplissage — le seul endroit où une implémentation par ailleurs juste se trompe.

```
cd WAC && g++ -std=c++17 -I. sha.cpp sha_test.cpp -o /tmp/sha_test && /tmp/sha_test
```

4.2 Décompression : `lznt1_test` et `xpress_test`

Vérifient les décompresseurs NTFS (LZNT1) et WOF (XPRESS Huffman) contre le *compresseur de Microsoft* : un fichier connu est compressé dans la VM par `RtlCompressBuffer`, unité par unité, et le harnais décompresse chaque unité puis la compare octet pour octet à l'original. Un décompresseur faux décode le plus souvent sans erreur : seule la comparaison exhaustive le révèle.

```
cd WAC
g++ -std=c++17 -I. lznt1.cpp lznt1_test.cpp -o /tmp/lznt1_test
g++ -std=c++17 -I. xpress.cpp xpress_test.cpp -o /tmp/xpress_test
/tmp/lznt1_test <original> <compressé.bin> <index.idx>
/tmp/xpress_test <original> <compressé.bin> <index.idx>
```

`index.idx` porte une ligne `<n> <taille originale> <taille compressée>` par unité ; une taille compressée nulle désigne une unité que Windows a renoncé à compresser.

4.3 Messages d'événements : `wevt_test` et `event_messages_test`

`wevt_test` vérifie la chaîne qui transforme un événement en phrase, sur les fichiers d'un vrai fournisseur : sa DLL et son satellite localisé.

```
cd WAC && g++ -std=c++17 -I. pe_resource.cpp wevt.cpp wevt_test.cpp -o /tmp/wevt_test
/tmp/wevt_test gpsvc.dll fr-FR/gpsvc.dll.mui "{aea1b4fa-97d1-45f2-a64c-4d69fffd92c9}" 1002
4001
```

event_messages_test fait de même à partir de la ruche SOFTWARE; il tourne sur la machine examinée ou dans la VM, seuls endroits où la ruche et les binaires des fournisseurs sont lisibles :

```
event_messages_test <ruche SOFTWARE> <guid> [id[:version] ...]
```

4.4 Journaux : evt_x_test

Lit un .evt_x hors de Windows, sous wine. Il sert à éprouver le décodeur sur des journaux que la VM ne produit pas : anciens, transférés, mal fermés, abîmés — et à comparer enregistrement par enregistrement à une implémentation indépendante (python-evt_x).

```
wine evt_x_test.exe "Z:/chemin/Security.evt_x"          # bilan par journal
wine evt_x_test.exe "Z:/chemin/Security.evt_x" 3        # + XML de 3 enregistrements
wine evt_x_test.exe "Z:/chemin/Security.evt_x" --dump   # un enregistrement par ligne
wine evt_x_test.exe --collect "Z:/tmp/arbre" "Z:/tmp/sortie" # chaîne complète
wine evt_x_test.exe --collect-memory "Z:/tmp/arbre" "Z:/tmp/sortie2" # même chaîne, en
    mémoire
cmp /tmp/sortie/events.json /tmp/sortie2/events.json    # doit être muet
```

La commande de compilation est dans le README, section *Testing the EVT_X parser outside Windows*.

4.5 Consigne : consigne_test

Rejoue la chaîne de production — empreintes, manifeste, copie vérifiée, jeu des ruches — sur des ruches données en argument, et vérifie ce qu'aucune compilation ne révèle : la consigne reste identique octet pour octet pendant que la copie de travail est modifiée, et le sceau porte l'empreinte réelle du manifeste.

```
wine consigne_test.exe "Z:/tmp/sortie" "Z:/chemin/SYSTEM" "Z:/chemin/ntuser.dat"
```

4.6 Lecture brute : raw_hive_test

Extrait un fichier en lisant directement le volume NTFS. À lancer sous Windows, en administrateur; le cycle VM l'utilise pour valider l'extraction d'une ruche.

```
raw_hive_test.exe [volume] [chemin] [sortie] [--fix] [--list] [--attrs]
```

--fix rend la ruche extraite chargeable, --list liste un répertoire, --attrs affiche les attributs \$MFT du fichier sans les interpréter.

4.7 Signatures : authenticode_test

Vérifie les catalogues d'un dossier puis rend un verdict par fichier ; ses verdicts se comparent à ceux de Get-AuthenticodeSignature sur les mêmes fichiers, dans la VM. Le seul écart qui compte : un fichier accepté par WAC et refusé par Windows.

```
authenticode_test --catalogs <dossier>  
authenticode_test --catalogs <dossier> --files <liste>
```

Les harnais de robustesse

5.1 Le principe

Un analyseur lit à des positions que le fichier lui-même déclare. Sur un fichier tronqué ou forgé, une lecture au-delà du tampon ne plante pas d'elle-même : elle lit la mémoire voisine et la publie, dans un JSON valide. Les harnais la font planter : chaque entrée est copiée contre une page mémoire interdite (PAGE_NOACCESS), une fois *finissant* contre elle (lectures au-delà de la fin), une fois *commençant* juste après elle (lectures en deçà du début, par exemple à une position négative tirée de la donnée). Un seul octet lu hors de l'entrée provoque une violation d'accès.

Pour chaque fichier donné, le harnais analyse :

- le fichier entier;
- ses troncatures (toutes les longueurs pour les petits fichiers);
- des copies dont des octets ont été remplacés au hasard, avec une graine fixe : un échec se reproduit à l'identique.

5.2 Raccourcis : lnk_test

```
cd build-windows/tests
WINEDLLOVERRIDES="prosys=n" wine lnk_test.exe a.lnk b.lnk ...
```

Sources de fichiers : le dossier Recent et le menu Démarrer de la VM, ou ceux d'une collecte (Users\<nom>\AppData\Roaming\Microsoft\Windows\Recent). Les noms accentués ou non latins passent mal dans la ligne de commande de wine : copier les fichiers sous des noms numérotés.

À retenir

WINEDLLOVERRIDES="prosys=n" fait charger le substitut de prosys.dll : wine n'implémente pas la fonction qui nomme les propriétés, et arrêterait le test dès le premier raccourci réel qui en porte.

5.3 Listes de sauts et Prefetch : parsers_test

```
parsers_test.exe jumplist-auto <fichier.automaticDestinations-ms> ...
```

```
parsers_test.exe jumplist-custom <fichier.customDestinations-ms> ...  
parsers_test.exe prefetch          <fichier.pf> ...
```

Les listes de sauts se testent sous wine, comme `lnk_test`. Les Prefetch se testent **dans la VM** : ils sont compressés (XPRESS Huffman), et seule la bibliothèque `ntdll` de Windows les décompresse. Le harnais décompresse chaque fichier avant de l'éprouver : c'est l'analyseur de WAC qu'il vise, pas le décompresseur de Windows.

Pour les 307 Prefetch d'une machine réelle (plus d'un million d'entrées), il faut compter environ deux heures dans la VM : lancer le test en tâche détachée, par lots, et suivre son journal. Exemple de script PowerShell, déposé dans la VM avec `qga.py write` :

```
$files = Get-ChildItem C:\wactest\pt\pf\*.pf | ForEach-Object { $_.FullName }  
$log = 'C:\wactest\pt\pf_results.txt'  
for ($i = 0; $i -lt $files.Count; $i += 20) {  
    $batch = $files[$i..([Math]::Min($i + 19, $files.Count - 1))]  
    Add-Content $log (& C:\wactest\pt\parsers_test.exe prefetch @batch 2>&1)  
    Add-Content $log ("batch " + $i + " exit=" + $LASTEXITCODE)  
}  
Add-Content $log 'DONE'
```

5.4 Référence

Résultats obtenus sur une machine réelle, chaque entrée analysée des deux côtés de la page interdite :

Fichiers	Entrées	Où
220 raccourcis	546 214	wine
48 listes de sauts automatiques	175 585	wine
23 listes de sauts personnalisées	79 036	wine
307 Prefetch	1 246 491	VM, environ 2 h

Recouper sur une collecte réelle

Une VM fraîchement installée ne produit ni fragmentation, ni listes de sauts anciennes, ni noms de fichiers exotiques. Plusieurs défauts ne se sont révélés que sur une collecte réelle, confrontée à une source indépendante :

- **listes de sauts / olefile** : les flux et le DestList, relus par un lecteur OLE indépendant, doivent donner exactement les mêmes entrées (numéros, dates, chemins). C'est ce qui a montré que les flux s'appellent « 1 » à « f » et non « 01 » à « 0f »;
- **raccourci / DestList** : dans une liste de sauts, la cible d'un raccourci et le chemin de son entrée DestList sont deux enregistrements du même fichier. Leur désaccord a révélé une page de code fausse (« d\x92orientation ») et un chemin Unicode ignoré;
- **événements / python-evtx** : comparaison enregistrement par enregistrement (evtx_test --dump);
- **signatures / Get-AuthenticodeSignature** (authenticode_test);
- **ancienne collecte / nouvelle** : décoder les mêmes fichiers avec le code courant et comparer, après normalisation, à la sortie d'une version précédente.

Attention

Une collecte réelle se traite en local. Si une ruche doit être lue avec le vrai offreg.dll, elle peut être copiée dans la VM de test, puis effacée aussitôt le test fini. Aucune copie ne reste dans le dossier de travail temporaire, et rien n'en entre dans le dépôt.

Lire un échec

7.1 Codes de sortie

Code	Signification
0	tout est passé
1	échec d'un contrôle, ou fichier d'entrée illisible
2	mauvais usage (arguments)
3	parsers_test : une analyse ne s'est pas terminée en 60 s (chien de garde)
0xC0000005 (-1073741819)	violation d'accès : lecture hors du tampon

7.2 Localiser une lecture hors tampon

Sous wine, une faute affiche la pile d'appels, avec des adresses relatives au programme (lnk_test+0x485bb). Sous Windows, parsers_test affiche lui-même l'entrée en cause et l'adresse fautive :

```
FAILED exception 0xC0000005 at exe+0x78b70, address read 0x159deb4200c:  
C:\wactest\pt\pf\CMD.EXE-0BD30981.pf truncated at 192 (guard after)
```

L'adresse se traduit en fonction avec la table des symboles du programme (adresse de base 0x140000000) :

```
x86_64-w64-mingw32-nm -C parsers_test.exe | grep -i " t " | sort > syms.txt  
# chercher le symbole dont l'adresse est juste inférieure à 0x140000000 + 0x78b70  
x86_64-w64-mingw32-objdump -d --start-address=0x140078b40 \  
--stop-address=0x140078b80 parsers_test.exe
```

L'instruction désassemblée donne le décalage lu (ici 0xcc(%r14)); il se rapproche alors, dans le code source, du champ lu à cette position. Rejouer ensuite l'entrée exacte (la troncature ou la corruption indiquée) pour vérifier la correction, puis toute la série.

7.3 Une boucle sans fin

Si `parsers_test` s'arrête sur « no end after 60 s », l'entrée nommée fait tourner un analyseur sans fin — typiquement une chaîne de secteurs cyclique dans un conteneur OLE. C'est un défaut au même titre qu'une lecture hors tampon : un fichier forgé suffirait à bloquer une collecte.